

Deepseek - CS 182 Homework 4

Question 5: Designing 2D Filters

User:

```
# As usual, a bit of setup

import time
import numpy as np
import matplotlib.pyplot as plt
import requests
import random
import torch
from PIL import Image
from scipy import ndimage

seed = 7
torch.manual_seed(seed)
random.seed(seed)
np.random.seed(seed)

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see
# http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

imagenet_mean = np.array([0.485, 0.456, 0.406])
imagenet_std = np.array([0.229, 0.224, 0.225])
```

Prompting Deepseek on the context of the upcoming tasks to ensure it fully understands the setup code and to have any helper functions that may be needed later on in memory.

```
def show_image(image, title=''):
    # image is [H, W, 3]
    # assert image.shape[2] == 3
    image = torch.tensor(image)
    plt.imshow(torch.clip((image * 255, 0, 255).int()))
    plt.title(title, fontsize=16)
    plt.axis('off')
    return

def show_multiple_images(images=[], titles=[]):
    assert len(images) == len(titles), "length of two inputs are not equal"
    N = len(images)
    # make the plt figure larger
    plt.rcParams['figure.figsize'] = [24, 24]

    for i in range(N):
        plt.subplot(1, N, i+1)
        show_image(images[i], titles[i])

    plt.show()

def rgb2gray(rgb):
    r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]
    gray = 0.2989 * r + 0.5870 * g + 0.1140 * b

    return gray
```

In this problem, you will design simple blurring and edge detection filters. Don't do anything to the code. Are you ready for the next task?

Deepseek:

Yes, I'm ready for the next task! I can see the setup code has been run and we have:

1. Imported necessary libraries (numpy, matplotlib, torch, etc.)
2. Set random seeds for reproducibility
3. Defined helper functions for displaying images
4. Loaded and displayed a grayscale image of a fox.

User:

Implement the `average_filtering` function using the following 3x3 kernel:

Prompting Deepseek to implement the averaging-filtering function as its first TODO task. My strategy is to prompt Deepseek, the entire helper function to ensure it knows to output the exact format of the helper function with the new code exactly where you want it to be.

Deepseek:

```
#####  
#####  
#                               #  
  
#####  
#####  
output = ndimage.convolve(image, kernel)  
    return output
```

This implementation:

1. Creates a kernel filled with ones using `np.ones((filter_size, filter_size))`
2. Normalizes the kernel by dividing by the total number of elements (`filter_size2`)
3. The resulting kernel has each element equal to $1/(\text{filter_size}^2)$, so the sum of all elements is 1.
4. Uses `ndimage.convolve` to apply the filter to the input image

The function will create averaging filters of different sizes (3x3, 6x6, 9x9) and display the results alongside the original image. As the kernel size increases, the blurring effect should become more pronounced.

Implement the `edge_detecting` function using a 3x3 Laplacian kernel:

Another strategy I use is send Deepseek screenshots of algebraic texts as it has no limit.

Deepseek:

I'll implement the edge detection function using the 3x3 Laplacian kernel as specified.

Another one-shot task.

This implementation:

- The Laplacian filter is particularly good at detecting fine details and edges in all directions since it's rotationally invariant.

Deepseek even went a head and mentioned what and why Laplacian filters are good for, allowing users to better learn what they're practical for.